

Programming a 2D Ising Model/Locating the Curie temperature

Contentshide

Beginning

8. Locating the Curie temperature

TASK 8a:

Polynomial fitting

TASK 8b:

Fitting in a particular temperature range

TASK 8c:

Finding the peak in C

TASK 8d:

PageDiscussion

ReadView sourceView historyTools

This is the eighth (and final) section of the Ising model experiment.

Main Page: Programming a 2D Ising Model

Previous section: 7. Determining the heat capacity

8. Locating the Curie temperature

You should have seen in the previous section that the heat capacity becomes strongly peaked in the vicinity of the critical temperature and that the peak became increasingly sharply peaked as the system size was increased — in fact, Onsager proved that in an infinite system the heat capacity should diverge at $T = T_C$. In our finite systems, however, not only does the heat capacity not diverge, the Curie temperature changes with system size! This is known as a *finite size effect*.

It can be shown, however, that the temperature at which the heat capacity has its maximum must scale according to $T_{C,L} = \frac{A}{L} + T_{C,\infty}$, where $T_{C,L}$ is the Curie temperature of an $L \times L$ lattice, $T_{C,\infty}$ is the Curie temperature of an infinite lattice, and A is a constant in which we are not especially interested. This scaling equation comes from expanding in the limit of large L .

TASK 8a:

Reference data has been generated for longer simulations than would be possible on the college computers in Python. Each file contains six columns: T , E , E^2 , M , M^2 , C (NOTE: these final five quantities are normalised per spin, unlike your Python data), and you can read them with the NumPy `loadtxt` function as before. For each lattice size, plot the reference data against your data. For *one* lattice size, save a SVG or PNG image of this comparison and add it to your report — add a legend to the graph to label which is which (check if you need a [reminder about use of legend](#)).

Polynomial fitting

To find the temperature at which the heat capacity and susceptibilities have their maxima, we are going to fit a polynomial to the data in the critical region. NumPy provides the useful `polyfit` and `polyval` functions for this purpose. The following code is written for the heat capacities - try to also repeat this for the susceptibility! The usage is as follows:

```
data = np.loadtxt("...") #assume data is now a 2D array containing two columns, T and C
T = data[:,0] #get the first column
C = data[:,1] # get the second column

#first we fit the polynomial to the data
fit = np.polyfit(T, C, 3) # fit a third order polynomial

#now we generate interpolated values of the fitted polynomial over the range of our function
T_min = np.min(T)
T_max = np.max(T)
T_range = np.linspace(T_min, T_max, 1000) #generate 1000 evenly spaced points between T_min and T_max
fitted_C_values = np.polyval(fit, T_range) # use the fit object to generate the corresponding values of C
```

TASK 8b:

Write a script to read the data from a particular file, and plot C and χ vs T , as well as a fitted polynomial. Try changing the degree of the polynomial to improve the fit — in general, it might be difficult to get a good fit! Attach a SVG or PNG image of an example fit to your report.

Fitting in a particular temperature range

Rather than fit to all of the data, we might want to fit in only a particular range. NumPy provides a very powerful way to index arrays based on certain conditions. For example, if we want to extract only those data points which are between a particular $T_{\min}$ and $T_{\max}$, we can use the following syntax:

```
data = np.loadtxt("...") #assume data is now a 2D array containing two columns, T and C
T = data[:,0] #get the first column
C = data[:,1] # get the second column

Tmin = 0.5 #for example
Tmax = 2.0 #for example

selection = np.logical_and(T > Tmin, T < Tmax) #choose only those rows where both conditions are true
peak_T_values = T[selection]
peak_C_values = C[selection]
```

TASK 8c:

Modify your script from the previous section. You should still plot the whole temperature range, but fit the polynomial only to the peak! You should find it easier to get a good fit when restricted to this region.

Finding the peak in C

Your fitting script should now generate two variables: `peak_T_range`, containing 1000 equally spaced temperature values between $T_{\min}$ and $T_{\max}$ (whatever you chose those values to be), and `fitted_C_values`, containing the fitted heat capacity at each of those points. Use the NumPy `max` function to find the maximum in C . If you store the maximum value of C in the variable `Cmax`, you can use the following notation to find the corresponding temperature:

```
Cmax = np.max(...)
Tmax = peak_T_range[fitted_C_values == Cmax]
```

TASK 8d:

Find the temperature at which the maximum in C occurs for each datafile that you were given or generated with Python. Make a text file containing two columns: the lattice side length (2,4,8, etc.), and the temperature at which C is a maximum. This is your estimate of T_C for that side length. Make a plot that uses the scaling relation given above to determine $T_{C,\infty}$. By doing a little research, you should be able to find the theoretical exact Curie temperature for the infinite 2D Ising lattice. How does your value compare to this? Are you surprised by how good/bad the agreement is? Attach a SVG or PNG image of this final graph to your report, and discuss briefly what you think the major sources of error are in your estimate.

This is the eighth (and final) section of the Ising model experiment.

Main Page: Programming a 2D Ising Model

Previous section: 7. Determining the heat capacity

Appearancehide

Text

- ☒ Small
- ☐ Standard
- ☐ Large

Width

- ☒ Standard
- ☐ Wide

